

How Worker Tracker is built, phase by phase

An offline-first farm notebook for workers, daily work, advances and wage payments. Each phase below builds on the one before it. The purple boxes show exactly where one phase plugs into the next.

Frontend

React 19 + TypeScript, Vite 8, hash routing

On the phone

localStorage per user, service worker for offline

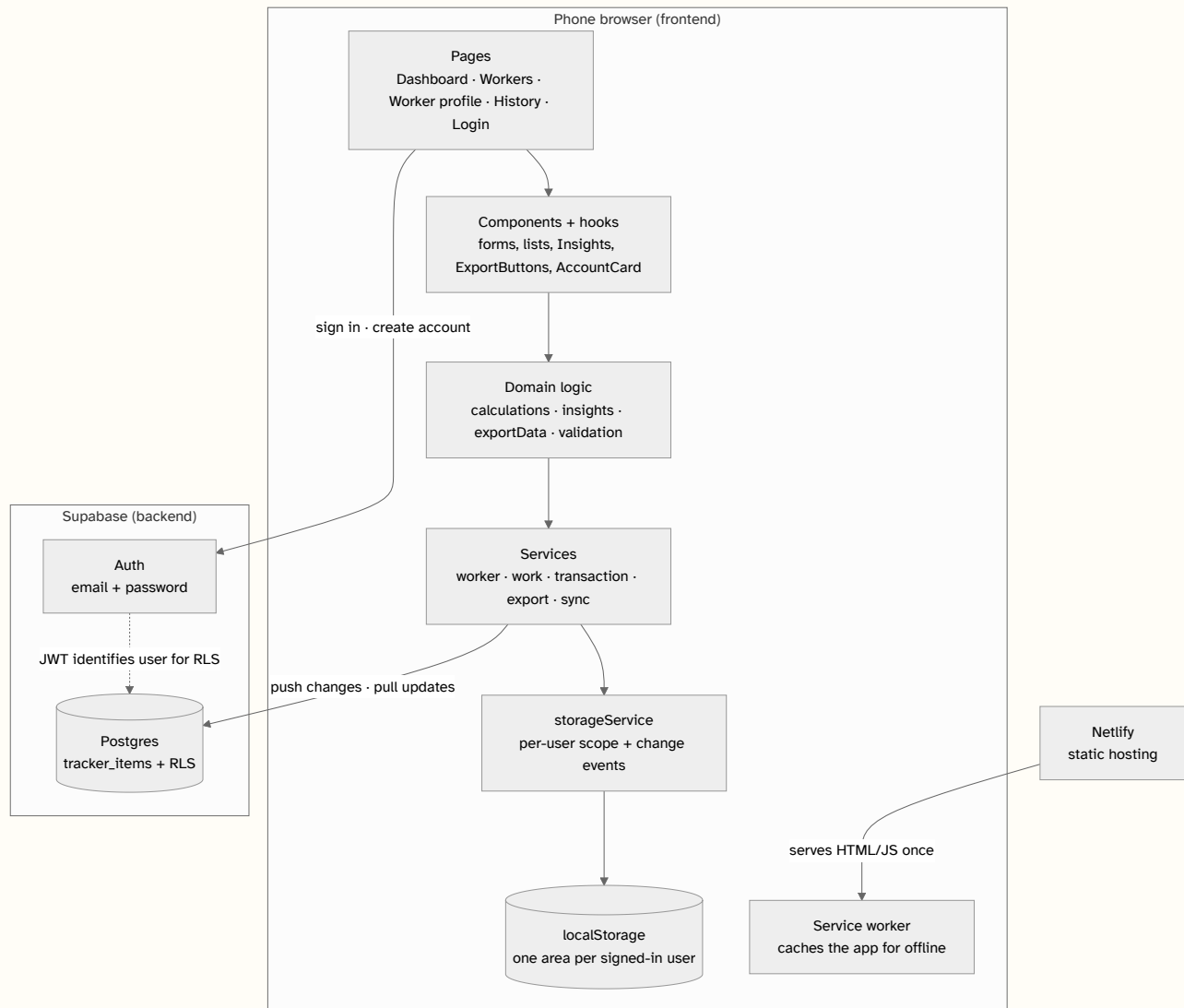
Backend

Supabase: Auth + Postgres table `tracker_items` with RLS

Hosting

Netlify static site `worker-tracker-farm`

The whole system at a glance



The phone is always the source the app reads from. Supabase is the shared copy that keeps phones in step.

The sign-in chain

How an email sign-in ends up deciding which rows a person can see. Each step hands one thing to the next.

Email

Farmer types an email and password. `normalizeEmail()` trims and lower-cases it; the password needs 6 or more characters.



Password

`signUp({ email, password })` creates the account (with a confirmation email if "Confirm email" is on); `signInWithPassword({ email, password })` signs in.



Supabase Auth

Checks the password and returns a session. The login token (JWT) carries the user id as `sub`. The session is saved on the phone, so the app opens offline.



user_id

`AuthProvider` reads `session.user.id`. The same email always maps to the same id, on every phone.



localStorage

`storageService.setScope("u:<user_id>")`: each person on a shared phone has a separate area.



sync

`syncService` uploads new, edited and deleted records under that `user_id` and downloads changes from other phones.



tracker_items

One row per worker, work day or payment: `user_id`, `collection`, `id`, `data`, `deleted`, `updated_at`.



RLS

Four policies allow SELECT, INSERT, UPDATE and DELETE only where `user_id = auth.uid()`, the id inside the login token. Tested on the real database by `tests/rls/tracker_items_rls.sql`.

Phases

1

Offline notebook on one phone BUILT

The original app. Everything is saved on the phone and works without internet. Money is stored as whole paise, and each work record keeps the wage it was saved with so later wage changes never rewrite history.

FRONTEND

`app/routes.tsx`

Hash routes like `#/workers/<id>`, so any static host works

`pages/*`

Dashboard, Workers, Worker profile, History

`hooks/useAppData.ts`

Loads workers, work and money together

LOGIC AND STORAGE

`domain/calculations.ts`

Balance = earned – (advances + wages paid)

`services/*Service.ts`

Validate, create, update, delete

`services/storageService.ts`

The only file that touches browser storage

`public/sw.js`

Caches the app shell for offline use

Connects to phase 2 and 3: the dashboard and exports reuse the same loaded data (`useAppData`) and the same balance maths (`summarizeWorker`), so numbers always agree across screens, Excel and PDF.

2

Dashboard insights BUILT

Below today's totals the dashboard shows the last 6 months of earnings against wages paid, and the workers you owe the most.

FRONTEND

`components/dashboard/Insights.tsx`

Bar chart, month table, "Most pending to pay" list

`pages/DashboardPage.tsx`

Adds Insights, export buttons and account card

LOGIC

`domain/insights.ts`

`monthlySeries()` and `topOwed()`, pure functions with tests

Connects to phase 1: reads only through `useAppData`, so it shows whatever the signed-in user's storage holds (phase 4) and refreshes when sync brings new data (phase 5).

Export to Excel and PDF BUILT

One Excel workbook (Workers, Work and Money sheets), one summary PDF of every account, and a statement PDF per worker with a running balance. The file builders load only when a button is tapped, so the app still opens fast.

FRONTEND

`components/export/ExportButtons.tsx`

Export Excel, Export PDF, Statement (PDF)

`services/exportService.ts`

Lazy-loads `write-excel-file` and

`jspdf`, then downloads

LOGIC

`domain/exportData.ts`

Turns records into rows in rupees, sorted by date

Connects to phase 1: uses `summarizeWorker` for balances. **Runs fully on the phone:** no server call, so exports work offline once the app has been opened online once.

Email login and per-user data BUILT

People sign in with their email and a password through Supabase Auth. "Create account" signs up; if email confirmation is on, they open the link in their inbox first. After sign-in, storage switches to that user's own area on the phone, so two people sharing a phone never see each other's data. Data saved before accounts existed moves to the first person who signs in.

FRONTEND

`auth/LoginPage.tsx`

Sign in and Create account; clear errors for wrong password, unconfirmed email, existing account, rate limits and no internet

`auth/emailAuth.ts`

Email clean-up and check, error messages

`auth/AuthProvider.tsx`

Shows login until signed in, then scopes storage and starts sync

`components/account/AccountCard.tsx`

Signed-in email, sync status, Sign out

BACKEND

Supabase Auth, Email provider

Checks the password, sends confirmation emails when switched on, and issues the login token (JWT); the session is saved on the phone, so it works offline

`services/supabaseClient.ts`

No keys means local-only mode with no login

Connects to phase 1: `storageService.setScope("u:<userId>:")` changes the storage keys only. Pages and services did not change. **Connects to phase 5:** the user id from the login is what sync uploads under and what the database rules check.

Cloud sync backend BUILT

Every save goes to the phone first, then uploads in the background. Other phones download changes when they open the app, come back online, or after a save. If two phones edit the same record, the later edit wins.

FRONTEND (SYNC ENGINE)

`services/syncService.ts`

Push: compares records with what was last uploaded and sends only new, edited or deleted ones

Pull: fetches rows changed since the last pull, 1,000 at a time, and merges by `updatedAt`

Triggers: 0.8 s after a save, when back online, when the app becomes visible

BACKEND (SUPABASE POSTGRES)

`tracker_items`

One row per worker, work record or payment: `user_id, collection, id, data, deleted, updated_at`

Row-level security

4 rules: a user can only read, add, edit or delete their own rows

Trigger

`tracker_items_touch`

The server sets `updated_at`, so a phone with a wrong clock can't hide changes

Connects to phase 4: listens to `storageService.onWrite`, and writes downloaded data with `{ silent: true }` so it isn't uploaded again. **Connects to phase 2:** when a pull changes data, `AuthProvider` remounts the screens so the dashboard shows it.

Deploy and live test LIVE

Every merge to `main` is built by GitHub Actions and published to GitHub Pages at `https://abhiram23-magnus.github.io/work-tracker/`. The build points at Supabase project `gzvsjkbpuguttmsodoiy` through `.env.production`. A second workflow can also deploy to Netlify once a `NETLIFY_AUTH_TOKEN` secret is added.

STILL TO DO

Add the site URL in Supabase →
Authentication → URL Configuration, and
decide on "Confirm email"

Run

```
tests/e2e/live.e2e.ts
```

against the production URL

ALREADY VERIFIED

Unit tests, lint and build pass

E2E tests (fake Supabase): sign-up, sign-in
errors, email confirmation, two users, two
phones, offline, logout

RLS on the real database: 14 SELECT /
INSERT / UPDATE / DELETE checks

Supabase security advisor reports no issues

Connects to every phase: GitHub Pages serves the built app once; after that the service worker (phase 1) runs it offline and Supabase (phases 4 and 5) handles accounts and sync.

How data moves

Saving a work day

1. `WorkEntryForm` calls `workService.create()`
2. Validation and wage calculation in `domain/`
3. `storageService.write()` saves to the user's area in `localStorage`
4. The write event schedules a sync 0.8 s later
5. `syncService` upserts the new row into `tracker_items`

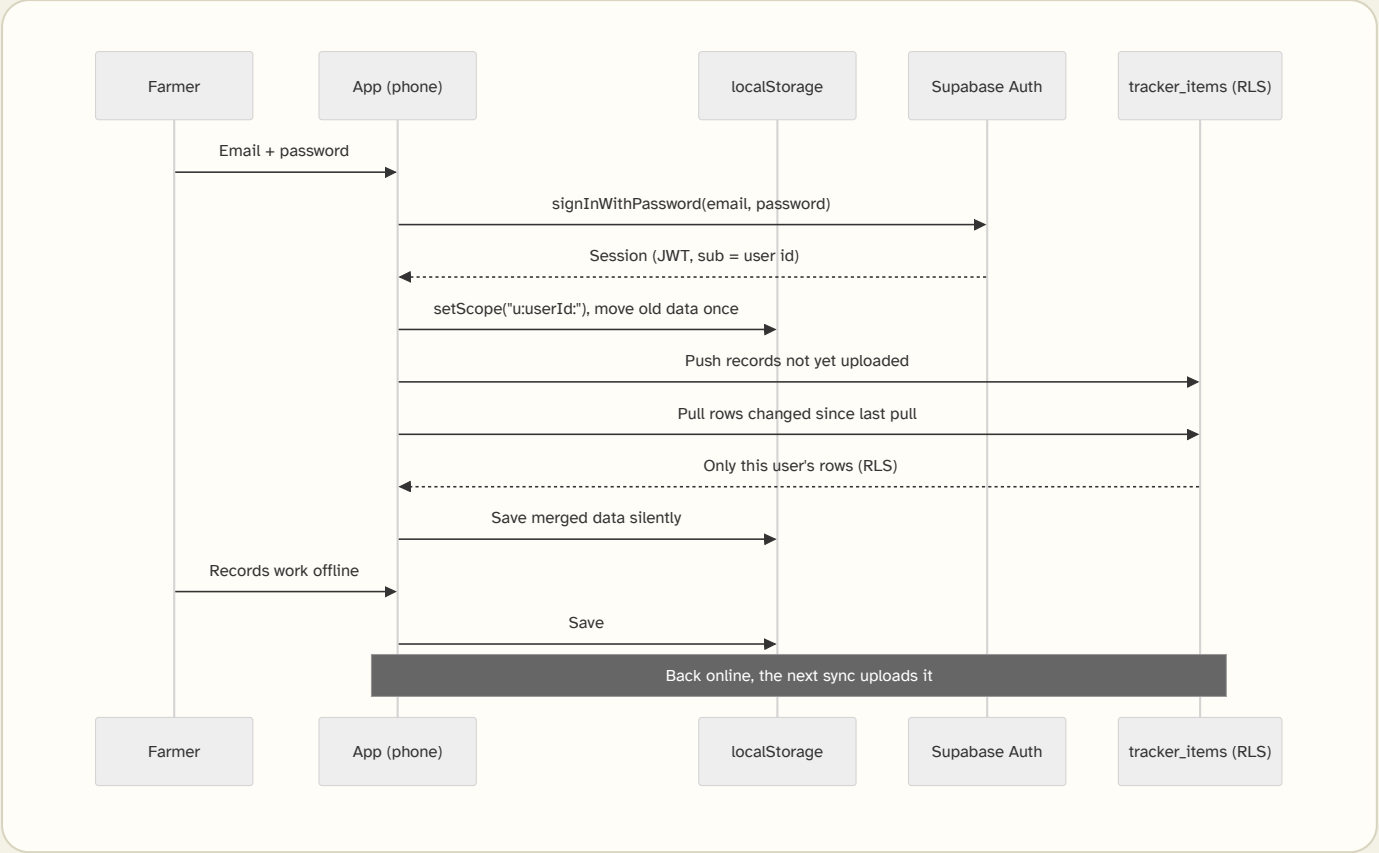
Opening on a second phone

1. `AuthProvider` restores the saved login
2. Storage switches to that user's area
3. Pull asks for rows newer than the last pull
4. `mergeRows()` keeps the newer edit and applies deletions
5. Screens remount and show the merged data

Exporting

1. Tap Export Excel or Export PDF
2. `buildExportTables()` or `buildStatement()` prepares rows
3. The Excel or PDF library loads on first use
4. The file downloads straight on the phone

Sign-in and sync sequence



Where each record lives

RECORD	ON THE PHONE	IN SUPABASE	KEY FIELDS
Worker	<code>...:u:<id>:workers</code>	<code>collection = 'workers'</code>	name, phone, workType, dailyWage (paise)
Work day	<code>...:u:<id>:workRecords</code>	<code>collection = 'workRecords'</code>	date, status, earnedAmount, dailyWage at save time

RECORD	ON THE PHONE	IN SUPABASE	KEY FIELDS
Advance or wage payment	...:u:<id>:transactions	collection = 'transactions'	date, type, amount (paise), note
Sync bookkeeping	...:setting:sync:<id>:*	not stored	what was last uploaded, last pull time